

Übungsaufgabe: *CloudConnect* — Skalierbare Container-Infrastruktur auf Azure

Lehrveranstaltung: Skalierbare Cloudarchitekturen (SKCLAR)

Abgabeform: Einzelarbeit

Lernziele

- Einfache Webanwendungen in Docker-Container verpacken und über eine private Registry (ACR) bereitstellen
- Multi-Container-Anwendungen lokal mit Docker Compose testen
- Eine vollständige Azure-Infrastruktur deklarativ per IaC (Bicep oder Terraform) provisionieren
- Ein Azure Application Gateway mit Path-based Routing als Layer-7-Einstiegspunkt konfigurieren
- Die Unterschiede zwischen Azure Load Balancer und Application Gateway erklären und begründen

KI-Nutzung: Der Einsatz von KI-Tools ist ausdrücklich erlaubt und erwünscht. Die generierten Artefakte müssen jedoch verstanden und erklärt werden können — es wird ein mündliches Abgabegespräch geben.

Szenario

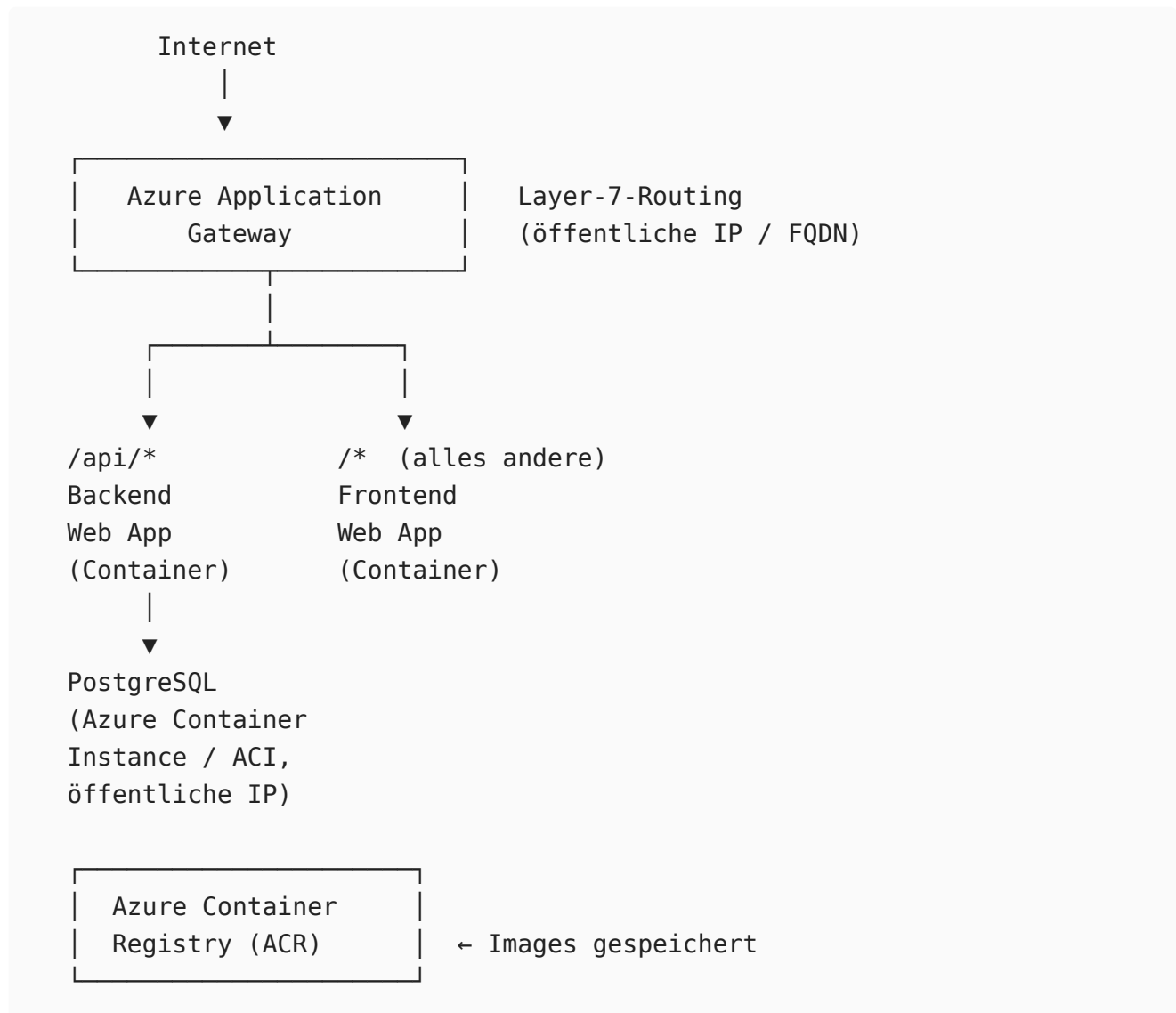
Sie bauen eine neue interne Plattform namens **CloudConnect** — ein einfaches schwarzes Brett, auf dem Nutzer kurze Nachrichten posten und lesen können. Die Funktionalität ist bewusst minimal gehalten; entscheidend ist die Infrastruktur dahinter.

Die Anwendung besteht aus drei Komponenten:

Komponente	Beschreibung	Technologie-Empfehlung
Frontend	Statische Webseite: zeigt Nachrichten, erlaubt neue einzutragen	HTML + Vanilla JS (fetch API)
Backend API	REST-API: mindestens GET /api/messages , POST /api/messages	Node.js/Express <i>oder</i> Python/FastAPI
Datenbank	Persistente Speicherung der Nachrichten	PostgreSQL

Die Anwendung muss **nicht** produktionsreif sein. Ein funktionierendes UI mit einem Textfeld und einer Nachrichtenliste reicht vollständig aus.

Geforderte Architektur



Alle Ressourcen (ACR, Web Apps, App Service Plan, Application Gateway, PostgreSQL als ACI) müssen per IaC provisioniert werden.

Aufgaben im Detail

Teil 1 — Anwendung & Containerisierung

1.1 Anwendungscode

Implementieren Sie Frontend und Backend minimal, sodass:

- Das Frontend eine HTML-Seite mit einem Eingabefeld und einer Nachrichtenliste zeigt

- Das Backend die Endpoints `GET /api/messages` und `POST /api/messages` bereitstellt
- Das Backend eine Verbindung zur PostgreSQL-Datenbank aufbaut (Verbindungsdaten über Umgebungsvariablen)

1.2 Dockerfiles

Erstellen Sie je ein `Dockerfile` für Frontend und Backend. Anforderungen:

- Multi-stage builds sind empfohlen, aber nicht verpflichtend
- Images müssen schlank sein (kein `node_modules` oder `__pycache__` im finalen Image)
- Konfiguration (DB-Host, Port, Passwort) ausschließlich über **Umgebungsvariablen** — keine Hardcoded Werte

1.3 Docker Compose (lokales Testen)

Erstellen Sie eine `docker-compose.yml`, die alle drei Dienste (Frontend, Backend, PostgreSQL) gemeinsam startet. Nach `docker compose up` muss die Anwendung unter `http://localhost` erreichbar sein.

Anforderungen an die Compose-Datei:

- Eigenes User-defined Bridge Network (kein Default-Bridge)
- Healthcheck für den Backend-Service
- Volume für PostgreSQL-Datenpersistenz
- `depends_on` mit `condition: service_healthy` wo sinnvoll

Teil 2 — Azure Container Registry & IaC-Grundstruktur

2.1 IaC-Werkzeug wählen

Entscheiden Sie sich für **Bicep** oder **Terraform**. Beide sind gleichwertig akzeptiert. Legen Sie eine klare Verzeichnisstruktur an:

```
infra/
├─ main.bicep          (oder main.tf)
├─ variables.bicep     (oder variables.tf)
├─ outputs.bicep       (oder outputs.tf)
└─ modules/            (optional, für saubere Struktur)
```

2.2 Azure Container Registry provisionieren

Provisionieren Sie per IaC eine ACR-Instanz (SKU: `Basic` reicht für die Übung). Anforderungen:

- ACR Admin Credentials (für einfaches `docker login`)

- Name und SKU als IaC-Variable, nicht hardcoded
- Output: Login-Server-URL

2.3 Images bauen und pushen

```
# Nach erfolgreichem IaC-Deploy der ACR:
az acr login --name <ihr-acr-name>
docker build -t <acr-login-server>/cloudconnect-frontend:v1 ./app/frontend
docker build -t <acr-login-server>/cloudconnect-backend:v1 ./app/backend
docker push <acr-login-server>/cloudconnect-frontend:v1
docker push <acr-login-server>/cloudconnect-backend:v1
```

2.4 App Service Plan + Web Apps for Containers

Provisionieren Sie per IaC:

- Einen Linux App Service Plan (SKU: B1 oder B2)
- Zwei Azure Web Apps for Containers (je eine für Frontend und Backend)
 - Konfiguration: Image aus ACR
 - App Settings für Umgebungsvariablen (DB-Verbindung etc.)
 - ACR Admin Credentials für den ACR-Pull

2.5 PostgreSQL als Azure Container Instance

Provisionieren Sie per IaC eine Azure Container Instance (ACI), die den postgres - Container ausführt:

- Image: postgres:15-alpine — wird **nicht direkt von Docker Hub** gezogen, sondern zuerst in die eigene ACR gespiegelt (Docker Hub Rate Limiting)
- Ressourcenanforderungen: 1 CPU, 1.5 GB RAM
- Umgebungsvariablen: POSTGRES_DB , POSTGRES_USER , POSTGRES_PASSWORD als IaC-Variablen
- DNS-Label konfigurieren, damit der Container über einen stabilen FQDN erreichbar ist (z.B. <ihr-prefix>-db.westeurope.azurecontainer.io)
- ACR-Credentials in der ACI-Konfiguration hinterlegen (imageRegistryCredentials)
- Output: FQDN des Datenbankcontainers

Schritt: Image in ACR spiegeln (einmalig nach ACR-Deploy)

```
az acr import --name <acr-name> \
  --source docker.io/library/postgres:15-alpine \
  --image postgres:15-alpine
```

Danach referenziert das IaC-Modul das Image als <acr-login-server>/postgres:15-alpine .

Hinweis zur Erreichbarkeit — bekannte Herausforderung:

Eine Azure Container Instance erhält standardmäßig eine **öffentliche IP-Adresse**. Da Azure App Service keinen direkten privaten Netzwerkpfad zu ACI hat (ohne zusätzliche VNet-Integration), verbindet sich das Backend über das **öffentliche Internet** mit der Datenbank auf Port 5432. In einer produktiven Umgebung wäre das nicht akzeptabel — der korrekte Weg wäre: ACI in einem delegierten VNet-Subnet deployen und für die Web App eine VNet-Integration aktivieren. Für diese Übung ist die öffentliche Variante ausdrücklich erlaubt.

Verwenden Sie ein starkes Datenbankpasswort, da Port 5432 öffentlich exponiert ist.

Achtung: Passwörter und Secrets dürfen **nicht** im IaC-Code hardcoded oder im Git-Repository eingecheckt sein. Verwenden Sie `.tfvars`-Dateien (in `.gitignore`) oder Bicep-Parameter-Dateien mit `@secure()`-Decorator.

Teil 3 — Application Gateway

3.1 Application Gateway per IaC

Provisionieren Sie ein Azure Application Gateway (SKU: `Basic`) mit folgender Konfiguration:

Komponente	Konfiguration
Frontend IP	Öffentliche statische IP-Adresse
Listener	Port 80, HTTP
Backend Pool 1	Frontend Web App (FQDN der App Service)
Backend Pool 2	Backend Web App (FQDN der App Service)
HTTP Settings	Pick hostname from backend target aktiviert
Health Probes	Je eine pro Backend Pool
Routing Rule 1	Path: <code>/api/*</code> → Backend Pool 2
Routing Rule 2	Path: <code>/*</code> (Default) → Backend Pool 1

3.2 End-to-End-Test

Nach vollständigem Deployment:

- Öffnen Sie `http://<application-gateway-public-ip>/` im Browser → Frontend muss laden
- Rufen Sie `http://<application-gateway-public-ip>/api/messages` auf → JSON-Antwort des Backends

3. Posten Sie über das Frontend eine Nachricht → Nachricht erscheint nach Reload in der Liste
4. Überprüfen Sie die Application Gateway Logs / Backend Health im Azure Portal

Teil 4 — Idempotenz-Nachweis

Führen Sie `bicep/terraform apply` ein **zweites Mal** aus (ohne Änderungen). Das Ergebnis muss sein:

- Terraform: 0 to add, 0 to change, 0 to destroy
- Bicep: Alle Ressourcen im Status `Succeeded`, keine neuen Deployments

Abgabe

Repository-Struktur (GitHub / GitLab — öffentlich oder Zugriff für Lehrenden)

```
cloudconnect/
├─ app/
│   ├── frontend/
│   │   ├── Dockerfile
│   │   └─ index.html (+ ggf. weitere Dateien)
│   └─ backend/
│       ├── Dockerfile
│       └─ main.js / main.py (+ ggf. weitere Dateien)
├─ docker-compose.yml
├─ infra/
│   ├── main.bicep / main.tf
│   ├── variables.bicep / variables.tf
│   ├── outputs.bicep / outputs.tf
│   └─ README.md ← Deploy-Anleitung (Pflicht!)
├─ screenshots/
│   └─ idempotenz_nachweis.png
└─ README.md ← Projektübersicht
```

Wichtig: Die `.gitignore` muss folgende Dateien ausschließen:

```
# Terraform
*.tfvars
.terraform/
*.tfstate
*.tfstate.backup
```

```
# Bicep
*.parameters.json    (falls Secrets enthalten)

# Allgemein
.env
```

Demovideo (3-4 Minuten)

Das Video muss folgende Punkte in dieser Reihenfolge zeigen:

1. `docker compose up` → Browser: Anwendung läuft lokal auf `localhost`
2. Kurze Code-Durchführung: Dockerfile und IaC-Struktur (max. 90 Sekunden)
3. `docker push` der Images in die ACR
4. `terraform apply` / `bicep deploy` läuft erfolgreich durch
5. Application Gateway URL im Browser → Frontend lädt
6. `/api/messages` -Endpoint antwortet (Browser oder Postman/curl)
7. Nachricht über Frontend posten → erscheint in der Liste
8. Screenshot Idempotenz-Nachweis zeigen und kurz erklären